

MongoDB

Complete Study Guide

Documents • Collections • CRUD • Aggregation • FastAPI Integration

Data Science & AI Engineering Prep

1. What is MongoDB and Why Does It Exist?

MongoDB is a NoSQL document database. Instead of storing data in rigid tables like SQL, MongoDB stores data as flexible JSON-like documents. Each document can have different fields, making it ideal for real-world data that varies in structure.

Real-Life Analogy:

Imagine a contact book. In SQL, every contact must have the exact same columns (name, phone, email). But real contacts vary — some have two phones, some have LinkedIn, some have addresses. MongoDB lets each contact be its own flexible object with whatever fields it needs.

1.1 SQL vs MongoDB — Core Mapping

SQL Term	MongoDB Term	Think of it as...
Database	Database	Same concept
Table	Collection	A folder of documents
Row	Document	One JSON-like object
Column	Field	A key in the JSON
JOIN	Embedded doc / \$lookup	Nest data inside document
CREATE TABLE	No command needed	Collection auto-created on first insert
Schema (required)	Schema (optional/flexible)	Documents in same collection can differ

1.2 Why Use MongoDB?

- Flexible Schema — each document can have different fields, no ALTER TABLE needed
- Horizontal Scaling — easy to distribute across many servers (sharding)
- Fast Prototyping — no schema setup, just start inserting documents
- Native JSON — APIs return JSON, MongoDB stores JSON — perfect match
- Rich Queries — powerful query language with operators, aggregation, indexing
- Great for AI/DS — storing LLM logs, chat histories, RAG documents, scraped data

2. Core Concepts — Documents, Collections, BSON

2.1 Documents

A document is the basic unit of data in MongoDB — like a row in SQL, but flexible. It is a JSON-like object with key-value pairs.

```
# A MongoDB document (stored as BSON internally)
{
  "_id": ObjectId("507f1f77bcf86cd799439011"), # Auto-generated unique ID
  "name": "Gurleen",
  "course": "Data Science",
  "score": 95,
  "skills": ["Python", "SQL", "MongoDB"], # Arrays are supported
  "address": { # Nested objects (embedded docs)
    "city": "Toronto",
    "country": "Canada"
  }
}
```

2.2 Collections

A collection is a group of documents — like a SQL table, but without a fixed schema. Documents in the same collection can have different fields.

```
# student_collection can have documents with different shapes:
{ "name": "Gurleen", "score": 95, "skills": ["Python"] }
{ "name": "Arjun", "grade": "A" } # Different fields - totally fine!
{ "name": "Sara", "score": 88, "city": "Mumbai" }
```

2.3 BSON — Binary JSON

- MongoDB stores data as BSON (Binary JSON) — the binary version of JSON
- BSON is faster to parse and search than plain text JSON
- BSON supports extra types that JSON doesn't: Date, ObjectId, int32, int64, Decimal128
- Every document gets an auto-generated `_id` field of type ObjectId
- ObjectId is a 12-byte unique identifier — globally unique across machines

3. Setting Up — PyMongo and Motor

3.1 Installation

```
# PyMongo — synchronous driver (for scripts, notebooks)
pip install pymongo

# Motor — async driver (for FastAPI integration)
pip install motor

# MongoDB Atlas — free cloud MongoDB (recommended for learning)
# Sign up at: mongodb.com/atlas
```

3.2 Connecting with PyMongo

```
from pymongo import MongoClient

# Option 1: Connect to local MongoDB
client = MongoClient("mongodb://localhost:27017/")

# Option 2: Connect to MongoDB Atlas (cloud)
client = MongoClient("mongodb+srv://username:password@cluster.mongodb.net/")

# Select database — auto-created on first insert
db = client["student_db"]

# Select collection — auto-created on first insert
collection = db["students"]

# KEY INSIGHT: MongoDB is lazy — nothing is created until you insert data!
```

3.3 Connecting with Motor (Async — for FastAPI)

```
from motor.motor_asyncio import AsyncIOMotorClient

# Same connection string, different client class
client = AsyncIOMotorClient("mongodb://localhost:27017/")
db = client["student_db"]
student_collection = db["students"]

# WHY Motor? FastAPI is async. PyMongo is sync (blocking).
# Motor is the async wrapper — won't freeze your API while querying.
```

4. CRUD Operations with PyMongo

4.1 CREATE — Insert Documents

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
collection = client["student_db"]["students"]

# INSERT ONE document
student = {
    "name": "Gurleen",
    "course": "Data Science",
    "score": 95,
    "skills": ["Python", "SQL", "MongoDB"]
}
result = collection.insert_one(student)
print(result.inserted_id)  # Returns the auto-generated ObjectId

# INSERT MANY documents at once
students = [
    {"name": "Arjun", "course": "AI", "score": 88},
    {"name": "Sara", "course": "MLOps", "score": 92},
    {"name": "Ravi", "course": "Data Science", "score": 76}
]
result = collection.insert_many(students)
print(result.inserted_ids)  # List of all inserted ObjectIds
```

4.2 READ — Find Documents

```
# FIND ONE — returns first match (or None)
student = collection.find_one({"name": "Gurleen"})
print(student)

# FIND ALL — returns a cursor (iterable)
all_students = collection.find()
for s in all_students:
    print(s["name"], s["score"])

# FIND WITH CONDITION — score greater than 80
top_students = collection.find({"score": {"$gt": 80}})
for s in top_students:
    print(s["name"], s["score"])

# FIND SPECIFIC FIELDS ONLY (projection)
# 1 = include, 0 = exclude, _id excluded with 0
collection.find({}, {"name": 1, "score": 1, "_id": 0})

# SORT results
collection.find().sort("score", -1)  # -1 = descending, 1 = ascending

# LIMIT results
collection.find().limit(5)  # Only return 5 documents
```

4.3 UPDATE — Modify Documents

```
# UPDATE ONE document - change Gurleen's score
collection.update_one(
    {"name": "Gurleen"},          # Argument 1: FILTER - which doc to find
    {"$set": {"score": 98}}       # Argument 2: UPDATE - what to change
)

# UPDATE MANY - add status field to all Data Science students
collection.update_many(
    {"course": "Data Science"},
    {"$set": {"status": "active"}}
)

# $inc - increment a value (useful for counters, scores)
collection.update_one(
    {"name": "Arjun"},
    {"$inc": {"score": 5}}        # score: 88 -> 93
)

# $push - add item to an array field
collection.update_one(
    {"name": "Gurleen"},
    {"$push": {"skills": "FastAPI"}}
)
```

4.4 DELETE — Remove Documents

```
# DELETE ONE document
collection.delete_one({"name": "Ravi"})

# DELETE MANY - remove all students with score below 80
collection.delete_many({"score": {"$lt": 80}})

# DELETE ALL documents in a collection
collection.delete_many({})      # Empty filter = match all - use carefully!
```

5. Query Operators

MongoDB uses special operators starting with \$ for conditions and updates. Here are the most important ones:

5.1 Comparison Operators (used in find/filter)

Operator	Meaning	Example
\$gt	Greater than	{"score": {"\$gt": 80}}
\$gte	Greater than or equal	{"score": {"\$gte": 90}}
\$lt	Less than	{"score": {"\$lt": 50}}
\$lte	Less than or equal	{"score": {"\$lte": 85}}
\$eq	Equal to	{"name": {"\$eq": "Gurleen"}}
\$ne	Not equal to	{"status": {"\$ne": "inactive"}}
\$in	Value is in a list	{"course": {"\$in": ["AI", "MLOps"]}}
\$nin	Value not in a list	{"course": {"\$nin": ["MLOps"]}}

5.2 Update Operators (used in update_one / update_many)

Operator	Meaning	Example
\$set	Set a field value	{"\$set": {"score": 98}}
\$inc	Increment a number	{"\$inc": {"score": 5}}
\$unset	Remove a field	{"\$unset": {"status": ""}}
\$push	Append to array	{"\$push": {"skills": "Docker"}}
\$pull	Remove from array	{"\$pull": {"skills": "Excel"}}
\$rename	Rename a field	{"\$rename": {"grade": "letter_grade"}}

5.3 Logical Operators

```
# $and - both conditions must be true
collection.find({"$and": [{"score": {"$gt": 80}}, {"course": "AI"}]})

# $or - at least one condition must be true
collection.find({"$or": [{"score": {"$gt": 90}}, {"course": "MLOps"}]})

# Shorthand AND - just put both in the same dict
collection.find({"score": {"$gt": 80}, "course": "AI"}) # Same as $and
```


6. Aggregation Pipeline

The aggregation pipeline is MongoDB's power tool for data analysis. Think of it as an assembly line — documents pass through stages, each stage transforms the data. It is the equivalent of SQL's GROUP BY, JOIN, and complex SELECT combined.

6.1 Pipeline Stages

Stage	Purpose (SQL Equivalent)
\$match	Filter documents — like SQL WHERE
\$group	Group and aggregate — like SQL GROUP BY
\$sort	Sort results — like SQL ORDER BY
\$limit	Limit number of results — like SQL LIMIT
\$project	Select/reshape fields — like SQL SELECT
\$unwind	Flatten array field into separate documents
\$lookup	Join with another collection — like SQL JOIN

6.2 Aggregation Example — Average Score Per Course

```
pipeline = [
    # Stage 1: Filter — only active students
    {"$match": {"status": "active"}},

    # Stage 2: Group — by course, calculate avg score and count
    {"$group": {
        "_id": "$course",                # Group by this field
        "avg_score": {"$avg": "$score"},  # Average score
        "total_students": {"$sum": 1},    # Count
        "highest_score": {"$max": "$score"} # Max score
    }},

    # Stage 3: Sort — by average score descending
    {"$sort": {"avg_score": -1}},

    # Stage 4: Limit — top 3 courses only
    {"$limit": 3}
]

results = collection.aggregate(pipeline)
for r in results:
    print(r)

# Output:
# {'_id': 'Data Science', 'avg_score': 92.5, 'total_students': 2, 'highest_score': 95}
```

```
# {'_id': 'AI', 'avg_score': 88.0, 'total_students': 1, 'highest_score': 88}
# {'_id': 'MLOps', 'avg_score': 76.0, 'total_students': 1, 'highest_score': 76}
```

7. Indexes — Making Queries Fast

Without an index, MongoDB scans every document to find a match (called a collection scan). This is slow for large collections. An index is like a book's index — instead of reading every page, you jump directly to the right page.

7.1 Creating Indexes

```
# Simple index on a single field
collection.create_index("email")

# Unique index — no two documents can have the same email
collection.create_index("email", unique=True)

# Compound index — index on two fields together
# 1 = ascending, -1 = descending
collection.create_index([("course", 1), ("score", -1)])

# Text index — for full-text search
collection.create_index([("description", "text")])

# List all indexes on a collection
print(list(collection.index_information()))
```

7.2 When to Create Indexes

- Always index fields you frequently use in `find()` queries
- Always index fields used in `sort()` — speeds up sorting dramatically
- Use unique indexes on fields like email, username that must be unique
- MongoDB automatically creates an index on `_id` — you don't need to create it
- Too many indexes slow down writes — only index what you actually query on

Without Index	With Index
Scans ALL documents one by one	Jumps directly to matching documents
$O(n)$ — time grows with collection size	$O(\log n)$ — much faster for large data
Slow for 1M+ documents	Fast even for 100M+ documents
No setup needed	Small write overhead (index must be updated on insert)

8. MongoDB + FastAPI Integration

FastAPI is async. PyMongo is synchronous (blocking). So we use Motor — the async MongoDB driver — for FastAPI integration. The code structure is nearly identical to PyMongo but with `async/await`.

8.1 Project Structure

```
mongo_fastapi/  
├── database.py      # MongoDB connection (Motor)  
├── models.py        # Pydantic models for request/response  
└── main.py          # FastAPI routes (CRUD endpoints)
```

8.2 database.py — Connection

```
from motor.motor_asyncio import AsyncIOMotorClient  
  
MONGO_URL = 'mongodb://localhost:27017'  
  
client = AsyncIOMotorClient(MONGO_URL)  
db = client['student_db']  
student_collection = db['students']
```

8.3 models.py — Pydantic Models

```
from pydantic import BaseModel  
  
# Request model — no id (MongoDB generates it)  
class StudentCreate(BaseModel):  
    name: str  
    course: str  
    score: float  
  
# Response model — includes id as string  
class StudentResponse(BaseModel):  
    id: str  
    name: str  
    course: str  
    score: float
```

8.4 main.py — Full CRUD API

```
from fastapi import FastAPI, HTTPException  
from bson import ObjectId  
from database import student_collection  
from models import StudentCreate, StudentResponse
```

```
app = FastAPI()

# Helper: convert MongoDB doc to dict (ObjectId -> string)
def doc_to_dict(doc) -> dict:
    return {
        "id":      str(doc["_id"]),    # ObjectId -> string for JSON
        "name":    doc["name"],
        "course":  doc["course"],
        "score":   doc["score"]
    }

# CREATE — POST /students
@app.post('/students', response_model=StudentResponse)
async def create_student(student: StudentCreate):
    result = await student_collection.insert_one(student.model_dump())
    new = await student_collection.find_one({"_id": result.inserted_id})
    return doc_to_dict(new)

# READ ALL — GET /students
@app.get('/students')
async def get_all_students():
    students = []
    async for doc in student_collection.find(): # async for with Motor
        students.append(doc_to_dict(doc))
    return students

# READ ONE — GET /students/{id}
@app.get('/students/{student_id}', response_model=StudentResponse)
async def get_student(student_id: str):
    doc = await student_collection.find_one({"_id": ObjectId(student_id)})
    if not doc:
        raise HTTPException(status_code=404, detail="Student not found")
    return doc_to_dict(doc)

# UPDATE — PUT /students/{id}
@app.put('/students/{student_id}', response_model=StudentResponse)
async def update_student(student_id: str, student: StudentCreate):
    await student_collection.update_one(
        {"_id": ObjectId(student_id)},
        {"$set": student.model_dump()}
    )
    updated = await student_collection.find_one({"_id": ObjectId(student_id)})
    if not updated:
        raise HTTPException(status_code=404, detail="Student not found")
    return doc_to_dict(updated)

# DELETE — DELETE /students/{id}
@app.delete('/students/{student_id}')
async def delete_student(student_id: str):
    result = await student_collection.delete_one({"_id": ObjectId(student_id)})
```

```
if result.deleted_count == 0:
    raise HTTPException(status_code=404, detail="Student not found")
return {"message": "Student deleted successfully"}
```

8.5 Key Differences: PyMongo vs Motor in FastAPI

Concept	PyMongo (sync)	Motor (async — FastAPI)
Import	<code>from pymongo import MongoClient</code>	<code>from motor.motor_asyncio import AsyncIOMotorClient</code>
Find one	<code>collection.find_one(...)</code>	<code>await collection.find_one(...)</code>
Insert	<code>collection.insert_one(...)</code>	<code>await collection.insert_one(...)</code>
Iterate cursor	<code>for doc in collection.find():</code>	<code>async for doc in collection.find():</code>
Update	<code>collection.update_one(...)</code>	<code>await collection.update_one(...)</code>
Delete	<code>collection.delete_one(...)</code>	<code>await collection.delete_one(...)</code>

9. When to Use MongoDB vs SQL

Use MongoDB when...	Use SQL (PostgreSQL/MySQL) when...
Data structure varies per record	Data is uniform and structured
You need flexible schema (evolving product)	You need strict data validation
Storing JSON from APIs or web scraping	Complex multi-table JOINS are common
Building real-time apps, IoT, event logs	Financial / transactional data (ACID compliance)
Horizontal scaling across many servers	Strong data consistency is critical
Fast prototyping — no schema migration needed	Reporting, analytics, BI tools
Storing LLM chat histories, RAG documents	Relational data with many foreign keys

9.1 MongoDB in AI / Data Science Projects

- Storing raw scraped data — each page has different structure, MongoDB handles it naturally
- Logging LLM inputs and outputs — flexible schema for different model responses
- Chat history storage — each conversation is a document with a messages array
- RAG pipelines — store chunks, embeddings, and metadata as documents
- User profiles in AI apps — different users have different preference fields
- Experiment tracking — log ML experiments with varying hyperparameters

10. Q&A Cards — Test Your Understanding

Q1 What is the difference between a Document and a Collection in MongoDB?	A Document is a single JSON-like record (equivalent to a row in SQL). A Collection is a group of documents (equivalent to a table). Unlike SQL tables, collections don't enforce a fixed schema — each document can have different fields.
Q2 Why does MongoDB use BSON instead of plain JSON?	BSON (Binary JSON) is the binary-encoded version of JSON. It is faster to parse and search than plain text JSON. It also supports extra data types that JSON doesn't have: ObjectId, Date, int32, int64, and Decimal128.
Q3 What are the two arguments passed to <code>update_one()</code> ?	Argument 1 is the FILTER — which document to find (e.g., <code>{'name': 'Gurleen'}</code>). Argument 2 is the UPDATE — what change to make using operators like <code>\$set</code> (e.g., <code>{'\$set': {'score': 98}}</code>). Think of it as: find this document, then apply this change.
Q4 Why do we use Motor instead of PyMongo with FastAPI?	FastAPI is built on async Python (asyncio). PyMongo is synchronous — it blocks the entire event loop while waiting for the database, which freezes all other requests. Motor is the async wrapper around PyMongo that works with FastAPI's event loop without blocking.
Q5 What does the <code>doc_to_dict()</code> helper function do and why is it needed?	MongoDB returns documents with <code>_id</code> as ObjectId — a special binary type. Pydantic (used by FastAPI for response models) cannot serialize ObjectId to JSON. <code>doc_to_dict()</code> converts ObjectId to a plain string with <code>str(doc['_id'])</code> and reshapes the dict to match the Pydantic response model.
Q6 What is the Aggregation Pipeline and what SQL concept is it similar to?	The aggregation pipeline processes documents through a series of stages (<code>\$match</code> , <code>\$group</code> , <code>\$sort</code> , <code>\$project</code>). Each stage transforms the data. It is MongoDB's equivalent of SQL's GROUP BY, combined with WHERE, ORDER BY, and SELECT in a single pipeline.
Q7 You have a collection of 10 million documents and query by email 500 times per second. What should you do?	Create an index on the email field: <code>collection.create_index('email', unique=True)</code> . Without an index, MongoDB scans all 10 million documents for each query ($O(n)$, very slow). With an index, it jumps directly to the match ($O(\log n)$, extremely fast).
Q8 What is the difference between <code>find_one()</code> and <code>find()</code> ?	<code>find_one()</code> returns a single document (the first match) or None if not found. <code>find()</code> returns a cursor — an iterable object that you loop through to get all matching documents. Use <code>find_one()</code> when you expect one result (by ID or unique field), <code>find()</code> when you expect multiple results.
Q9 When would you choose	Choose MongoDB for: storing LLM chat histories (flexible message structures), RAG document storage (chunks with varying metadata),

MongoDB over a SQL database for an AI project?

logging model inputs/outputs (different models have different response shapes), and web scraping (each page has different fields). Use SQL when you need strict relationships between entities, complex JOINS, or ACID transactions.

**Q10
What does async for mean in Motor queries and why is it needed?**

Motor's find() returns an async cursor — you cannot iterate it with a regular for loop. You must use async for to consume it. This is because each iteration awaits the next batch of documents from the database asynchronously, allowing FastAPI to handle other requests while waiting for MongoDB data.

11. MongoDB Cheat Sheet

Quick Reference — Most Common Operations

```

from pymongo import MongoClient
from motor.motor_asyncio import AsyncIOMotorClient
from bson import ObjectId

# — Connect —————
client = MongoClient("mongodb://localhost:27017/")
collection = client["mydb"]["mycollection"]

# — Create —————
collection.insert_one({"name": "Alice", "score": 90})
collection.insert_many([{"name": "Bob"}, {"name": "Carol"}])

# — Read —————
collection.find_one({"name": "Alice"})
collection.find() # All documents
collection.find({"score": {"$gt": 80}}) # With condition
collection.find({}, {"name": 1, "_id": 0}) # Specific fields only
collection.find().sort("score", -1) # Sort descending
collection.find().limit(10) # Limit 10 results
collection.count_documents({}) # Count all documents

# — Update —————
collection.update_one({"name": "Alice"}, {"$set": {"score": 95}})
collection.update_many({"score": {"$lt": 60}}, {"$set": {"status": "fail"}})
collection.update_one({"name": "Bob"}, {"$inc": {"score": 5}})

# — Delete —————
collection.delete_one({"name": "Bob"})
collection.delete_many({"score": {"$lt": 50}})

# — Index —————
collection.create_index("email", unique=True)
collection.create_index([("course", 1), ("score", -1)])

# — Aggregation —————
pipeline = [
    {"$match": {"status": "active"}},
    {"$group": {"_id": "$course", "avg": {"$avg": "$score"}}},
    {"$sort": {"avg": -1}},
    {"$limit": 5}
]
list(collection.aggregate(pipeline))

# — ObjectId conversion (for FastAPI) —————
str(doc['_id']) # ObjectId -> string (for JSON response)
ObjectId(student_id) # string -> ObjectId (for queries)

```

Key Operators at a Glance

Category	Operators	Use in
Comparison	\$gt, \$gte, \$lt, \$lte, \$eq, \$ne	find() filter
List check	\$in, \$nin	find() filter
Logical	\$and, \$or, \$not	find() filter
Set field	\$set	update_one/many
Increment	\$inc	update_one/many
Remove field	\$unset	update_one/many
Array append	\$push	update_one/many
Array remove	\$pull	update_one/many
Pipeline filter	\$match	aggregate()
Pipeline group	\$group	aggregate()
Pipeline sort	\$sort	aggregate()
Pipeline select	\$project	aggregate()

End of MongoDB Complete Guide